L11 — Multi-Dimensional Arrays: Representation and Applications

Unit: 1 | **Chapter:** Sorting Techniques | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Represent 2D and 3D arrays in memory using row-major and column-major order
- Derive address formulas for elements in multi-dimensional arrays
- Implement basic matrix operations
- Identify real-world applications of multi-dimensional arrays

---

1. What are Multi-Dimensional Arrays?

A **multi-dimensional array** extends the concept of a 1D array to multiple dimensions. The most common is the **2D array** (matrix), but 3D and higher-dimensional arrays exist.

```
1D Array:       A[5]
2D Array:       A[3][4]   (3 rows, 4 columns)
3D Array:       A[2][3][4] (2 layers, 3 rows, 4 columns)
```

In memory, all multi-dimensional arrays are stored as **1D sequences** — either row by row or column by column.

---

2. 2D Array Memory Representation

2.1 Row-Major Order (C, C++, Java, Python)

Elements are stored row by row:

```
A[0][0], A[0][1], A[0][2], A[0][3],
A[1][0], A[1][1], A[1][2], A[1][3],
A[2][0], A[2][1], A[2][2], A[2][3]
```

Address formula (0-indexed, m rows, n columns): $$\text{Addr}(A[i][j]) = \text{BA} + (i \times n + j) \times \text{size}$$

2.2 Column-Major Order (Fortran, MATLAB, Julia)

Elements are stored column by column:

```
A[0][0], A[1][0], A[2][0],
A[0][1], A[1][1], A[2][1],
...
```

Address formula (0-indexed, m rows, n columns): $$\text{Addr}(A[i][j]) = \text{BA} +

(j \times m + i) \times \text{size}$$

2.3 Worked Examples

Example 1 (Row-Major): BA = 3000, `int` (4 bytes), A[4][5] — find A[2][3]: $$ \text{Addr} = 3000 + (2 \times 5 + 3) \times 4 = 3000 + 13 \times 4 = 3052$$

Example 2 (Column-Major): BA = 2000, `int` (4 bytes), A[3][4] — find A[1][2]: $$ \text{Addr} = 2000 + (2 \times 3 + 1) \times 4 = 2000 + 7 \times 4 = 2028$$

---

3. 3D Array Representation

For array `A[p][m][n]` (p layers, m rows, n columns):

Row-Major Address (0-indexed): $$\text{Addr}(A[i][j][k]) = \text{BA} + (i \times m \times n + j \times n + k) \times \text{size}$$

Example: BA = 1000, `float` (4 bytes), A[2][3][4] — find A[1][2][3]: $$= 1000 + (1 \times 3 \times 4 + 2 \times 4 + 3) \times 4 = 1000 + (12 + 8 + 3) \times 4 = 1000 + 92 = 1092$$

---

4. Matrix Operations

4.1 Matrix Addition — O(m × n)

```
def matrix_add(A, B, m, n):
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            C[i][j] = A[i][j] + B[i][j]
    return C
```

4.2 Matrix Multiplication — O(m × n × p)

Multiply A (m × n) by B (n × p) to get C (m × p):

```
def matrix_multiply(A, B, m, n, p):
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):          # Dot product of row i and col
                C[i][j] += A[i][k] * B[k][j]
    return C

# Example: 2×3 matrix × 3×2 matrix = 2×2 matrix
A = [[1, 2, 3], [4, 5, 6]]
B = [[7, 8], [9, 10], [11, 12]]
C = matrix_multiply(A, B, 2, 3, 2)
# C = [[58, 64], [139, 154]]
```

Time: $O(m \times n \times p)$ | **Space:** $O(m \times p)$

4.3 Matrix Transpose — $O(m \times n)$

```
def transpose(A, m, n):
    T = [[0] * m for _ in range(n)]   # T is n×m
    for i in range(m):
        for j in range(n):
            T[j][i] = A[i][j]
    return T
```

4.4 In-Place Transpose (Square Matrix) — $O(n^2)$

```
def transpose_inplace(A, n):
    for i in range(n):
        for j in range(i + 1, n):    # Only upper triangle
            A[i][j], A[j][i] = A[j][i], A[i][j]
```

---

5. Common 2D Array Patterns

5.1 Row Sum and Column Sum

```
def row_col_sums(matrix, m, n):
    row_sums = [sum(matrix[i]) for i in range(m)]
    col_sums = [sum(matrix[i][j] for i in range(m)) for j in range(n)]
    return row_sums, col_sums
```

5.2 Rotate Matrix 90° Clockwise (In-Place)

```
def rotate_90(matrix, n):
    # Step 1: Transpose
    transpose_inplace(matrix, n)
    # Step 2: Reverse each row
    for i in range(n):
        matrix[i].reverse()
```

5.3 Diagonal Elements

```
def print_diagonals(matrix, n):
    main_diag = [matrix[i][i] for i in range(n)]
    anti_diag = [matrix[i][n - 1 - i] for i in range(n)]
    return main_diag, anti_diag
```

---

6. Real-World Applications

Domain	Application	Data Structure Used
Computer Graphics	Pixel grid, image filters	2D array (bitmap)
Machine Learning	Weight matrices, datasets	2D/3D arrays (tensors)

Graph Theory	Adjacency matrix	2D array ($n \times n$)
Game Development	Game board (chess, Sudoku)	2D array
Scientific Computing	Finite element analysis	3D arrays
Spreadsheets	Table data	2D array
NLP	Embedding matrices	2D array (vocab $\times$ dim)

---

7. Performance Considerations

Cache Performance: Row-Major Access

In C/Python, iterating in **row-major order** is cache-friendly:

```
# Cache-friendly: accessing row by row (same as storage order)
for i in range(m):
    for j in range(n):
        process(A[i][j])

# Cache-unfriendly: column by column (jumps in memory)
for j in range(n):
    for i in range(m):
        process(A[i][j])
```

For large matrices, cache-friendly access can be **5-10$\times$ faster** due to cache line utilization.

---

8. Sparse Matrices

When most elements are 0 (e.g., adjacency matrices of sparse graphs), storing the full 2D array wastes memory.

Alternatives for sparse matrices:

- **COO (Coordinate format):** Store (row, col, value) for non-zeros
- **CSR (Compressed Sparse Row):** Efficient for row operations
- **Dictionary of keys:** `{(i, j): val}` for flexible access

(Covered in depth in L15 — Sparse Matrices)

Summary

- Multi-dimensional arrays are stored linearly in memory using **row-major** (C/Python) or **column-major** (Fortran/MATLAB) order.
- Address formula for A[i][j] in row-major: BA + ($i \times n$ + j) $\times$ size.
- Core operations: addition $O(m \times n)$, multiplication $O(m \times n \times p)$, transpose $O(m \times n)$.
- Cache-friendly access patterns follow the storage order (row-major in Python).
- Sparse matrices require specialized storage formats to avoid wasting memory.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 4 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.4 (Springer, 2020)